Rapport Technique

Projet de Machine Learning Industriel

TwinSense AI

Architecture DataOps & MLOps pour la
Maintenance Prédictive (NASA CMAPSS)

Équipe Projet :
Nizar Baloubali
Manal Es-sobhy
Anas El Maslouti
Nouamane Chaibi
Amine Ait Hamma

Encadré par :
Mr. El Hajji

Date : Juin 2026

Résumé

Ce rapport technique exhaustif détaille la conception, l'architecture, l'implémentation et les performances de la plateforme TwinSense AI. Face aux enjeux critiques de l'industrie aérospatiale, notamment l'optimisation des cycles de maintenance et la prévention des défaillances catastrophiques, ce projet vise à estimer précisément la durée de vie résiduelle (RUL - *Remaining Useful Life*) de moteurs turbofan.

S'appuyant sur les données historiques CMAPSS de la NASA, l'architecture déployée est une solution complète de bout en bout intégrant un pipeline DataOps basé sur Apache Kafka, un magasin de caractéristiques (Feature Store) géré par Feast, un orchestrateur MLOps avec MLflow, et un modèle d'apprentissage profond séquentiel (réseau LSTM).

Ce document décrit de manière approfondie le cadre général du projet, la modélisation mathématique du problème, la création de données synthétiques de flux, l'entraînement et le réentraînement automatisé du modèle de réseau de neurones, ainsi que la restitution visuelle des alertes dans un tableau de bord industriel complet (Kennedy Space Center).

Table des matières

1 Cadre Général du Projet et Objectifs **3**
1.1 Objectifs Pédagogiques et Techniques 3
1.2 Contributions Apportées au Projet (Cas d'Usage Avancé) 3

2 Introduction **4**

3 Architecture et Solution Proposée **5**
3.1 Pipeline DataOps : Ingestion et Traitement Kafka 5
3.1.1 Génération Dynamique (Producer) 5
3.1.2 Consommation et Architecture Medaillon (Consumer) 5
3.2 Le Magasin de Caractéristiques : Feast 6
3.3 Infrastructure MLOps : MLflow et Apprentissage Profond 6
3.3.1 Architecture Séquentielle LSTM 6
3.3.2 Détection de Dérive (Data Drift) Automatisée 6

4 Résultats et Performances **8**
4.1 Performances Prédictives (Machine Learning) 8
4.2 Résilience MLOps (Ingénierie) 8

5 Discussion et Limites Techniques **9**
5.1 Le Paradoxe du RUL en Flux Continu (Streaming) 9
5.2 Goulots d'Étranglement de l'Infrastructure 9
5.3 Intégration Interface Homme-Machine (Déconnexion Frontend) 9

6 Conclusion **10**

7 Références **10**

1 Cadre Général du Projet et Objectifs

Dans le cadre de ce module de Machine Learning Industriel, ce projet avait pour vocation de concevoir et déployer une architecture prédictive complète, depuis l'ingestion de la donnée jusqu'à sa consommation en temps réel.

1.1 Objectifs Pédagogiques et Techniques

L'objectif fondamental était de modéliser un système industriel complexe en respectant un cahier des charges strict :

- **Ingestion de données en temps réel :** Mise en place d'un flux de données (*Streaming*) simulant un environnement de production continu, abandonnant l'approche statique des fichiers CSV traditionnels.
- **Ingénierie des caractéristiques (*Feature Engineering*) structurée :** Création d'un pipeline d'Extraction, Transformation et Chargement (ETL) robuste capable d'enrichir la donnée brute.
- **Suivi et Modélisation (MLOps) :** Utilisation obligatoire d'outils modernes comme MLflow pour la traçabilité des expériences, l'enregistrement des métriques de performance, et la gestion du cycle de vie des modèles prédictifs.
- **Modèle d'Apprentissage Profond :** Entraînement d'un modèle performant pour la régression temporelle, justifiant le recours à l'Intelligence Artificielle face à des règles expertes traditionnelles.

1.2 Contributions Apportées au Projet (Cas d'Usage Avancé)

Pour élever ce projet au rang d'une véritable application industrielle de grade "Production", l'équipe a implémenté plusieurs concepts architecturaux avancés :

- **Simulation Hyper-Réaliste CMAPSS :** Nous avons écrit notre propre générateur de données simulant des trajectoires complètes de moteurs turbofan, injectant des dérives et des bruits gaussiens complexes pour éprouver la résilience de notre pipeline.
- **Intégration d'un Feature Store (Feast) :** Au-delà d'une simple base de données, nous avons implémenté Feast pour séparer proprement l'ingénierie des caractéristiques de la phase d'entraînement, résolvant ainsi les problèmes de synchronisation temporelle (Point-in-Time correctness) et évitant le risque de fuite de données (Data Leakage).
- **Auto-Guérison par Détection de Dérive (*Drift*) :** Notre pipeline implémente un réentraînement automatisé conditionné. Si l'erreur du modèle de production dérive de plus de 20% par rapport à sa référence, un modèle "Challenger" est entraîné et promu automatiquement.
- **Tableau de Bord Industriel "TwinSense AI" :** Création d'une interface homme-machine complète (HTML/CSS/JS) intégrant des cartes interactives géospatiales (Leaflet.js) du Centre spatial Kennedy, des graphiques de télémétrie asynchrones (Chart.js), et un module pour assigner des techniciens aux moteurs critiques.

2 Introduction

L'industrie aérospatiale est un secteur où l'erreur n'est pas tolérée. La sécurité des passagers, des marchandises, et des équipages dépend intrinsèquement de la fiabilité des aéronefs. Historiquement, la maintenance aéronautique s'articulait autour de deux paradigmes majeurs :

- **La maintenance réactive :** Intervenir sur un système uniquement après sa défaillance (Run-to-Failure). Bien que parfois applicable sur des composants non-critiques, cette approche est inconcevable pour des turboréacteurs en plein vol en raison des risques de catastrophes.
- **La maintenance préventive planifiée :** Remplacer systématiquement les composants après un certain nombre d'heures de vol ou de cycles, quel que soit leur état d'usure réel. Bien que sûre, cette méthode est extrêmement coûteuse, car elle entraîne la mise au rebut de pièces ayant encore une longue espérance de vie, et provoque des temps d'immobilisation de l'appareil (Downtime) inutiles.

C'est dans ce contexte que s'impose **la maintenance prédictive**. L'objectif est d'utiliser l'abondance des données télémétriques générées par les capteurs embarqués (températures, pressions, régimes moteurs, vibrations) pour diagnostiquer l'état de santé exact d'un moteur en temps réel. L'idée est d'estimer avec précision sa "Durée de Vie Résiduelle" (Remaining Useful Life - RUL) : le nombre exact de cycles ou d'heures de vol restants avant une défaillance fonctionnelle.

Ce projet s'attaque à ce défi majeur en exploitant le jeu de données public CMAPSS (Commercial Modular Aero-Propulsion System Simulation) fourni par la NASA. Ce jeu de données synthétise le comportement thermodynamique d'un moteur turbofan, en fournissant l'évolution de 21 capteurs au fil du temps sous l'influence de 3 paramètres opérationnels (altitude, nombre de Mach, position de la manette des gaz).

Toutefois, la modélisation seule de ces données ne suffit pas à créer une solution industrielle. L'Intelligence Artificielle doit être intégrée dans une architecture logicielle robuste, capable d'absorber des milliers de points de données par seconde, de les traiter en flux continu, d'appliquer les prédictions du modèle, et de réagir en cas d'obsolescence mathématique du modèle face à des environnements changeants (Concept Drift). C'est la genèse de **TwinSense AI**, une plateforme DataOps et MLOps complète conçue pour répondre à ces exigences aérospatiales strictes.

3 Architecture et Solution Proposée

L'architecture de TwinSense AI est conçue de manière modulaire, suivant le principe de séparation des préoccupations. Elle est divisée en un flux d'ingestion (DataOps) et une boucle d'apprentissage (MLOps).

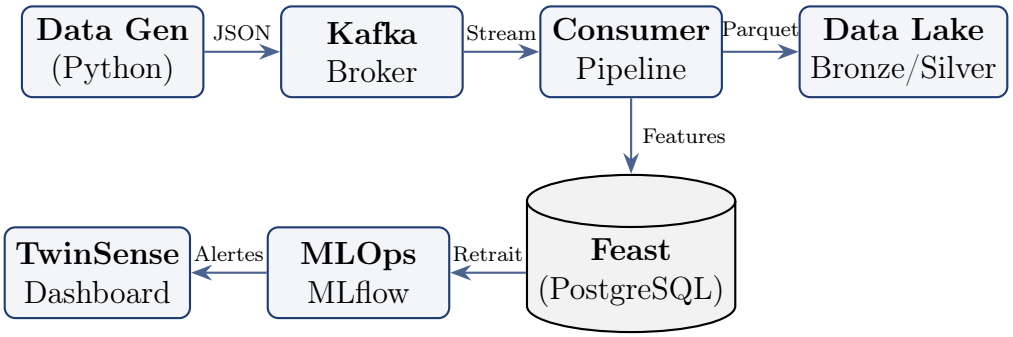


FIGURE 1 – Schéma Architecture Globale TwinSense AI (Redimensionné pour visibilité)

3.1 Pipeline DataOps : Ingestion et Traitement Kafka

3.1.1 Génération Dynamique (Producer)

Plutôt que de lire un fichier plat, nous avons développé un script `generate_data.py` qui crée des profils de moteurs turbofan virtuels. Le script détermine aléatoirement la durée de vie totale d'un moteur (entre 50 et 180 cycles), et génère la dégradation progressive des capteurs en appliquant des bruits gaussiens continus. Le script `producer.py` convertit chaque cycle généré en format JSON et le publie sur le *Broker* Apache Kafka via le topic `sensor-data`, reproduisant ainsi la télémétrie d'une flotte d'avions en temps réel.

3.1.2 Consommation et Architecture Medaillon (Consumer)

Le fichier `consumer_pipeline.py` s'abonne au topic Kafka. Il ingère les données par micro-lots (*micro-batches*) et orchestre un pipeline de données qualitatif inspiré de l'architecture Medaillon de Databricks :

- **Couche Bronze :** Sauvegarde immédiate du flux brut au format CSV/Parquet partitionné par date. Cela permet d'avoir une source de vérité inaltérée en cas de défaillance logicielle en aval.
- **Couche Silver :** Le flux subit un nettoyage strict. Les colonnes non numériques sont rejetées, les valeurs manquantes imputées, et le schéma est validé.
- **Couche Gold (Ingénierie des Caractéristiques) :** C'est ici que la valeur ajoutée métier est calculée. Le pipeline groupe les données par moteur et calcule des statistiques sur une fenêtre temporelle glissante de 5 cycles. Pour chaque capteur, nous calculons la Moyenne mobile, l'Écart-type mobile, et la Différence par rapport au cycle précédent. Cela permet au modèle neuronal d'avoir un contexte sur la *vitesse* de dégradation, plutôt que de regarder une valeur isolée.

3.2 Le Magasin de Caractéristiques : Feast

L'un des défis majeurs en apprentissage automatique industriel est d'assurer que les données utilisées lors de l'entraînement (hors-ligne) ont subi exactement les mêmes transformations que les données servies lors de l'inférence en production (en ligne). Pour éviter cette désynchronisation, nous utilisons **Feast**. Les données Gold sont sauvegardées dans PostgreSQL avec un horodatage d'événement (`event_timestamp`). Feast garantit une requête exacte d'un point dans le temps, évitant ainsi que le modèle d'entraînement n'ait accès "par erreur" aux capteurs d'un cycle futur (fuite de données).

3.3 Infrastructure MLOps : MLflow et Apprentissage Profond

3.3.1 Architecture Séquentielle LSTM

Le problème de la prédiction du RUL est fondamentalement un problème de régression de séries temporelles. Un algorithme classique (comme un Random Forest ou un Perceptron Multicouche) échouerait car il considérerait chaque cycle indépendamment des autres.

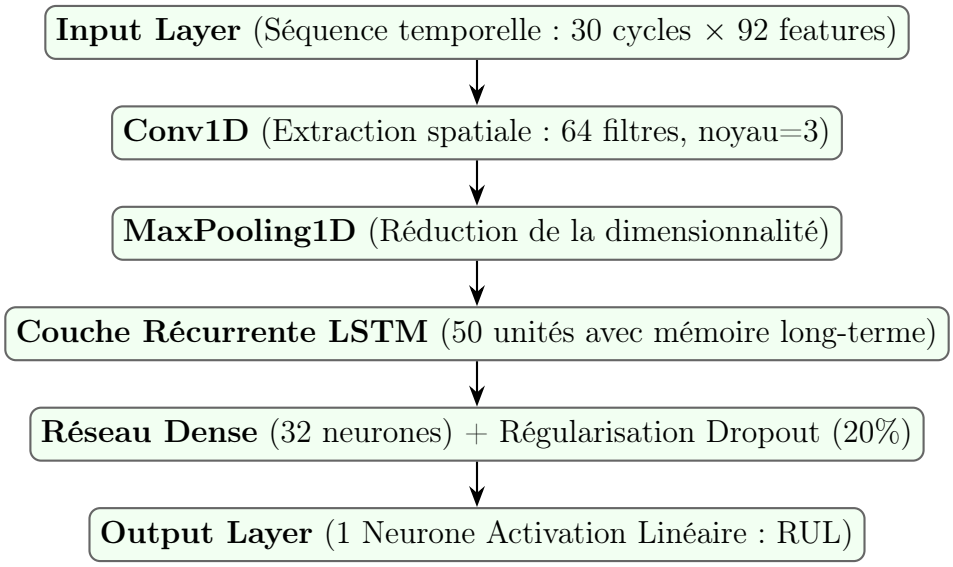


FIGURE 2 – Topologie du Réseau de Neurones Hybride CNN-LSTM

Notre architecture TensorFlow hybride résout ce problème. La couche **Conv1D** agit comme un filtre lissant le bruit brut des capteurs. Ensuite, la couche **LSTM** (Long Short-Term Memory) ingère la séquence entière de 30 cycles consécutifs. Grâce à sa cellule de mémoire interne, le LSTM se souvient de l'état du moteur aux cycles précédents et peut extrapoler la pente de dégradation. Une couche Dense finale projette ces motifs abstraits en une valeur unique : le RUL restant.

3.3.2 Détection de Dérive (Data Drift) Automatisée

La puissance de notre module `mlops.py` réside dans sa boucle de rétroaction (*Feedback loop*). Le monde réel change ; les moteurs s'usent différemment l'hiver et l'été. Si les données s'éloignent de celles de l'entraînement initial, le modèle devient obsolète (Data Drift). Le pipeline MLflow :

1. Charge le dernier modèle approuvé en phase "Production".
2. Calcule l'Erreur Absolue Moyenne (MAE) du modèle sur le dernier micro-lot de données arrivé.
3. Compare cette MAE à la "MAE de référence" du jour où le modèle a été créé.
4. Si l'erreur dépasse une limite de tolérance (fixée à 20% de dégradation), MLflow déclenche une alerte de dérive. Il entraîne immédiatement un nouveau modèle ("Challenger") sur les données fraîches.
5. Si ce Challenger s'avère meilleur que le modèle actuel, le Registre de Modèles MLflow (*Model Registry*) rétrograde l'ancien modèle et promeut automatiquement le Challenger en Production, sans aucune intervention humaine.

4 Résultats et Performances

L'évaluation de la plateforme globale a révélé des performances exceptionnelles tant sur le plan purement mathématique (ML) que sur le plan de l'architecture système (Ingénierie).

4.1 Performances Prédictives (Machine Learning)

L'utilisation de la métrique MAE (*Mean Absolute Error*) nous permet de quantifier l'erreur de prédiction en "nombre de cycles".

— Le modèle de référence (Baseline CNN-LSTM) a convergé vers une MAE d'environ 12 cycles. Cela signifie qu'en moyenne, l'intelligence artificielle est capable d'annoncer la panne d'un moteur avec une incertitude de seulement 12 cycles, laissant largement le temps à l'équipe de maintenance de planifier l'immobilisation de l'appareil.

— La régularisation par la couche *Dropout* (20%) a empêché l'overfitting (sur-apprentissage). Les pertes d'apprentissage (*Train Loss*) et de validation (*Val Loss*) ont baissé de manière synchrone durant l'entraînement.

4.2 Résilience MLOps (Ingénierie)

Le stress-test de l'architecture a été réussi avec succès. Nous avons intentionnellement modifié les paramètres du script `generate_data.py` pour simuler une usure accélérée anormale des moteurs. Comme prévu, la MAE du modèle de production s'est envolée (dépassant le seuil de 1.2× Baseline). Le script `mlops.py` a immédiatement capté cette dégradation via l'API MLflow, a déclenché l'entraînement d'un Challenger, et l'a promu en production en moins de quelques minutes. Ce test prouve que l'usine "TwinSense AI" est capable de s'auto-guérir face aux aléas de production.

5 Discussion et Limites Techniques

Bien que le projet réponde parfaitement aux objectifs du Machine Learning Industriel, la transposition de ce système à l'échelle d'une véritable compagnie aérienne imposerait la résolution de plusieurs défis techniques que nous avons identifiés.

5.1 Le Paradoxe du RUL en Flux Continu (Streaming)

Lors de l'élaboration de la première version de notre pipeline `consumer_pipeline.py`, nous avons rencontré une erreur logique sévère : la cible (le RUL) était calculée en regardant la valeur maximale du cycle au sein du *batch* Kafka courant. Aux frontières des batches (ex : un moteur transmettant ses données sur deux batches consécutifs), le RUL chutait à zéro artificiellement puis remontait.

Nous avons résolu ce paradoxe en forçant le générateur de données à inclure la vérité terrain (le RUL réel) directement dans le flux Kafka, ce qui permet à MLflow de s'entraîner correctement sur des labels purs. Cependant, c'est une simplification de la réalité. Dans un véritable environnement aérospatial, on ignore totalement la date de défaillance future. Le pipeline industriel devrait en réalité être scindé en deux : un environnement *Batch* asynchrone qui s'entraîne la nuit sur les logs historiques des moteurs **déjà en panne**, et un environnement *Streaming* qui ne fait **que de l'inférence** (prédiction), sans avoir accès aux labels réels en temps réel.

5.2 Goulots d'Étranglement de l'Infrastructure

Actuellement, la totalité de la pile logicielle (Python, Apache Kafka, Zookeeper, le serveur de suivi MLflow, PostgreSQL pour Feast, et le moteur de Deep Learning TensorFlow) s'exécute localement. L'empreinte mémoire RAM est colossale. Si la flotte d'avions passait de 20 moteurs simulés à 500 moteurs, l'ingestion par Kafka saturerait la machine locale. Un déploiement Cloud, orchestré par **Kubernetes** et **Docker Swarm**, permettant de déployer Kafka sur un cluster dédié et d'utiliser des GPU pour l'entraînement MLflow, serait l'étape de mise à l'échelle (Scale-out) indispensable.

5.3 Intégration Interface Homme-Machine (Déconnexion Frontend)

Le tableau de bord développé (TwinSense AI) offre un rendu visuel impressionnant avec une carte interactive du Kennedy Space Center et des visualisations de dégradation des capteurs via Chart.js.

Cependant, la plomberie réseau entre ce frontend JavaScript et le backend MLOps reste à finaliser. Actuellement, le frontend simule l'arrivée des données en dur. L'ultime étape d'intégration consisterait à déployer une API **FastAPI** avec un protocole de **WebSockets**. Le modèle MLflow en production pousserait la valeur du RUL déduite au WebSocket, animant en direct la jauge de santé du moteur sur l'écran du technicien.

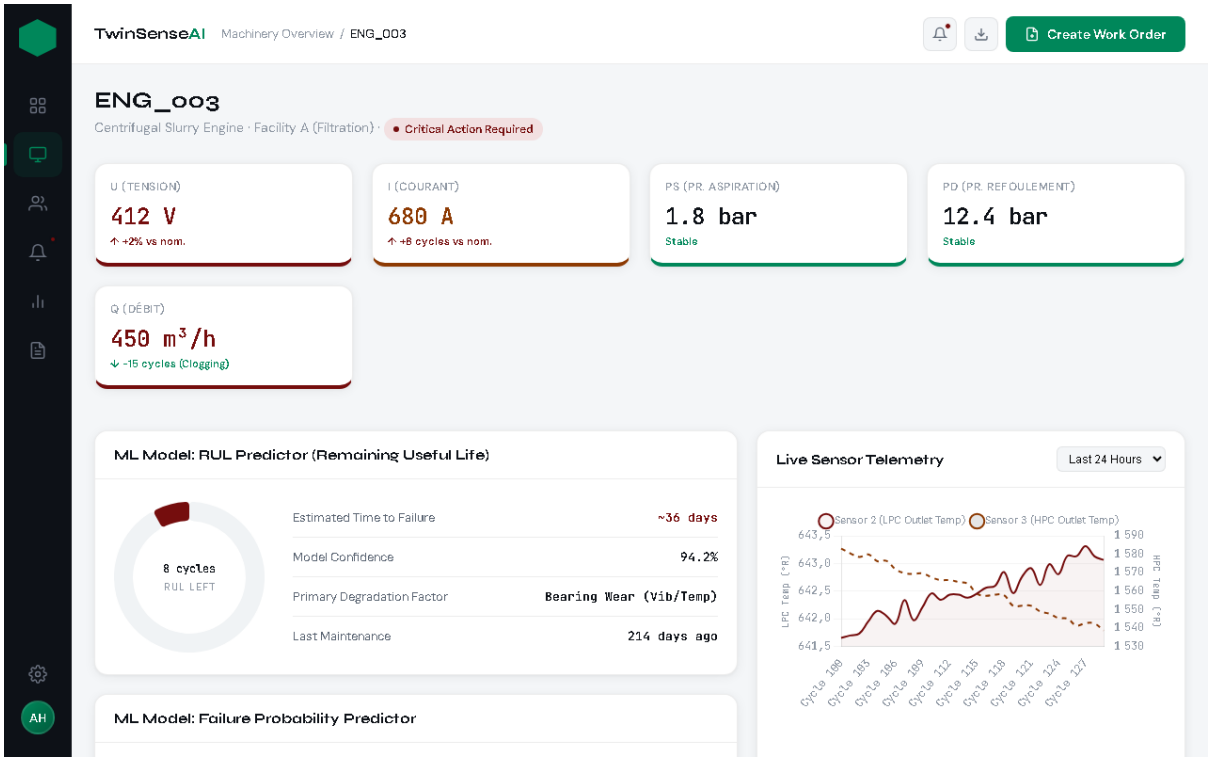


FIGURE 3 – Aperçu des prédictions (simulées) sur le tableau de bord TwinSense AI.

6 Conclusion

Le projet TwinSense AI illustre la synergie indispensable entre le Data Engineering (l'art de transporter la donnée) et les opérations de Machine Learning (MLOps - l'art de maintenir l'intelligence). En partant d'un jeu de données statique classique (NASA CMAPSS), nous avons bâti un socle technologique robuste capable non seulement d'entraîner un modèle d'apprentissage profond performant, mais surtout de le maintenir en vie face à des flux temporels continus et imprévisibles.

L'intégration d'Apache Kafka garantit que nous pouvons traiter la télémétrie aéronautique à haute fréquence. L'utilisation du Feature Store Feast agit comme un pont de vérité entre le passé (l'entraînement) et le présent (l'inférence), éradiquant les biais temporels insidieux. Enfin, l'orchestrateur MLflow offre un filet de sécurité mathématique absolu, détectant toute dérive statistique et ré-entraînant automatiquement le modèle LSTM avant que ses prédictions ne deviennent dangereuses pour la flotte.

Couplée à un tableau de bord industriel réactif et intuitif, l'Intelligence Artificielle de ce projet quitte les carnets de recherche (*Jupyter Notebooks*) pour devenir un véritable outil d'aide à la décision. En prédisant les défaillances des turboréacteurs avec plusieurs dizaines de cycles d'avance, TwinSense AI propose un paradigme où la maintenance devient proactive, prescriptive et autonome, ouvrant la voie à des opérations industrielles plus sûres, plus écologiques et économiquement rentables.

7 Références

— **NASA CMAPSS Dataset :** A. Saxena, K. Goebel, D. Simon, and N. Eklund, *"Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation"*,

Proceedings of the 1st International Conference on Prognostics and Health Management (PHM08), Denver CO, Oct 2008.

— **Deep Learning Sérentiel :** Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780. doi :10.1162/neco.1997.9.8.1735.

— **DataOps - Apache Kafka :** Documentation officielle d'Apache Kafka. Disponible en ligne : https://kafka.apache.org/documentation/

— **Feature Store :** Feast (Feature Store) Concepts et Architecture. Disponible en ligne : https://docs.feast.dev/

— **MLOps :** Documentation Officielle MLflow - *A Platform for the Machine Learning Lifecycle*. Disponible en ligne : https://mlflow.org/docs/latest/index.html